Université Cotes d'Azur/EUR DS4H **Novembre 2022**
Master 1 Electronique
Enseignement : Composants logiques programmables
Documents de cours/TD/TP autorisés

CONTROLE (1H30)

Exercice 1 : Question de cours

1.1 – Le rôle d'un FPGA est de réaliser des accélérateurs matériels. Que cherche-t-on à améliorer par rapport à une exécution sur un processeur ?

1.2 - Lister les différentes étapes qui permettent de concevoir un système sur un composant logique programmable et préciser leur rôle.

1.3 - Pour déterminer le nombre de cellules élémentaires (CE), citer les approches utilisées dans le cas des CPLDs, des FPGAs avec LUT puis avec MPX.

1.4 - Lors des phases de placement et de routage, que cherche-t-on à optimiser ? Ces deux phases peuvent elles être réalisées séparément ou de manière conjointe ?

1.5 – Expliquer comment est calculé le temps de réponse sur un FPGA.

Exercice 2 : Synthèse sur FPGA

On se propose dans le cadre de cet exercice de réaliser une unité de calcul (32 bits) pour la cryptographie intégrant les fonctions : application d'un masque M, la fonction XOR, des décalages. Les équations par étage pour la sortie S_i et le calcul de la retenue Ri sont données ci-dessous :

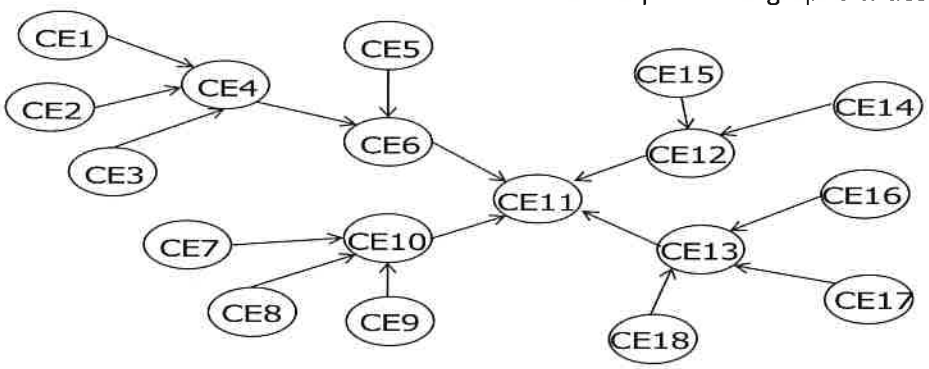
$S_i = /Sel_1./Sel_0.M_i.A_i + /Sel_1.Sel_0.(A_i \text{ XOR } B_i) + Sel_1./Sel_0 . A_{i-1} + Sel_1.Sel_0 . A_{i-2}$

2.1 -Calculer le nombre de cellules nécessaires pour effectuer la synthèse de ce composant pour les familles suivantes : CPLD MAX7000 (8 produits), FPGA avec 4-LUT puis MPX4->1.

2.2 - Calculer avant routage le temps de réponse : Tcellmax7000 = 3ns, Tcell_4LUT = 1.5 ns, Tcell_mpx= 1ns.

Exercice 3 : Routage

Considérons des cellules élémentaires interconnectées comme indiquer sur le graphe ci-dessous.



3. 1- Effectuer le placement et le routage pour un FPGA type MESH.

3.2 – Evaluer le temps de réponse avec Tcell= 3 ns, T_{SM} = 1ns.

3.3 – Rependre les questions précédentes pour un FPGA de type MESH de clusters de 8 CEs: Tcell=3ns, $T_{inter_cluster}$=3ns.

Exercice 4 (Relatif aux TPs) : Gestion d'un écran vidéo

On souhaite réaliser sur FPGA un système de détection de contour. Pour y parvenir, l'image est transformée en une image en niveau de gris, puis passe par un filtre sobel en x et un filtre sobel en y. L'image finale est reconstituée à partir des sorties des 2 filtres sobel.

4.1 - Lister les process pour réaliser ce système et indiquer les fonctionnalités réalisées.

4.2- Est-il possible d'améliorer les performances de ce système ? dans le cas positif, expliquer comment et le gain attendu.

1) On cherche à accélérer la vitesse d'exécution ← que cherche-t-on à améliorer / à une exécution d'un processeur ?

2) Les étapes de conception d'un système programmable :

1) Spécification / modélisation :
Définir les fonctions à réaliser (combinatoire, séquentielle, contrainte de temps, taille...)

2) écriture des equations logiques / description VHDL :

3) Synthèse logique :
CPLD : nombre de cellules (macro-cells)
FPGA : décomposition en LUT ou en cell MPx

4) Placement :
Affectation des cellules logiques aux blocs (LAB, CLB)

5) Routage :
connexion des cellules via les interconnexions programmables :
switch-box (mesh)
fast track (Altera)
PIA (CPLD Max 7000)

6) Génération du bit stream :
Programmation SRAM / EPROM / antifusible selon le composant.

3) Les approches pour déterminer le nombre de cellules

CPLD :
nbre de fct FI à réaliser → si P (ou : à N) alors NC = 1 (nbre d'entrée de la porte / bit présente dans les cellules)
si P > N alors NC = [(P-N)/(N-1)] + 1

FPGA avec LUT :
Représentation en arbre and / OR
Decomp OR si nécessaire
Regroupement des AND pour k entrées

FPGA avec MPx :
décomposition expression en arbre binaire de MuPx
suivre la décomp = nbre de cellules.

1-4- Que cherche-t-on à optimiser au placement / Routage :

- Réduire le temps de réponse en minimisant :
- le temps des interconnexions.
- nombre de switch / Box traversés.
- la distance entre cellules critiques.
- Placement et routage : séparé ou conjoint ?
- théoriquement séparé.
- Optimiser ensemble car le placement influence le routage (en pratique).

5) Temps de réponse en FPGA :

$$T_{total} = T_{cell} + T_{routing}.$$

Exo 2 :

A) CPLD : N = 8

P = 4 & 8 = N ⇒ 1 CE / bit ⇒ 32 cell / bit.

1 cell

T = 1 ns

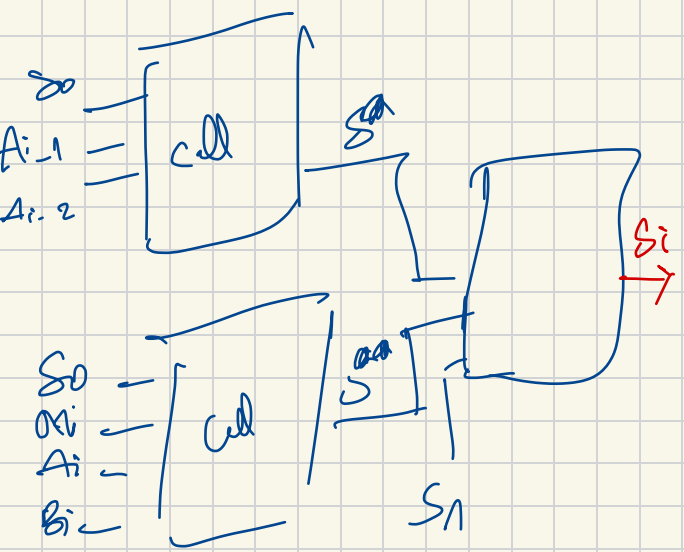
FPGA avec LUT

7 entrées (Sel0, Sel1, Mi, Ai, Ai-1, Ai-2, Bi)

$$S = \overline{Sel_1}\cdot\overline{sel_0}\cdot M_i A_i + \overline{Sel_1}\cdot sel_0\cdot(A_i \oplus B_i) + sel_1\cdot\overline{Sel_0}\,A_{i-1} + sel_1\,sel_0 A_{i-2}$$

$$S_1(S_0 A_{i-1} + S_0 A_{i-2}) + \overline{S_1}(S_0 M_i A_i + (S_0(A_i \oplus B_i))$$

S^* S^{**}



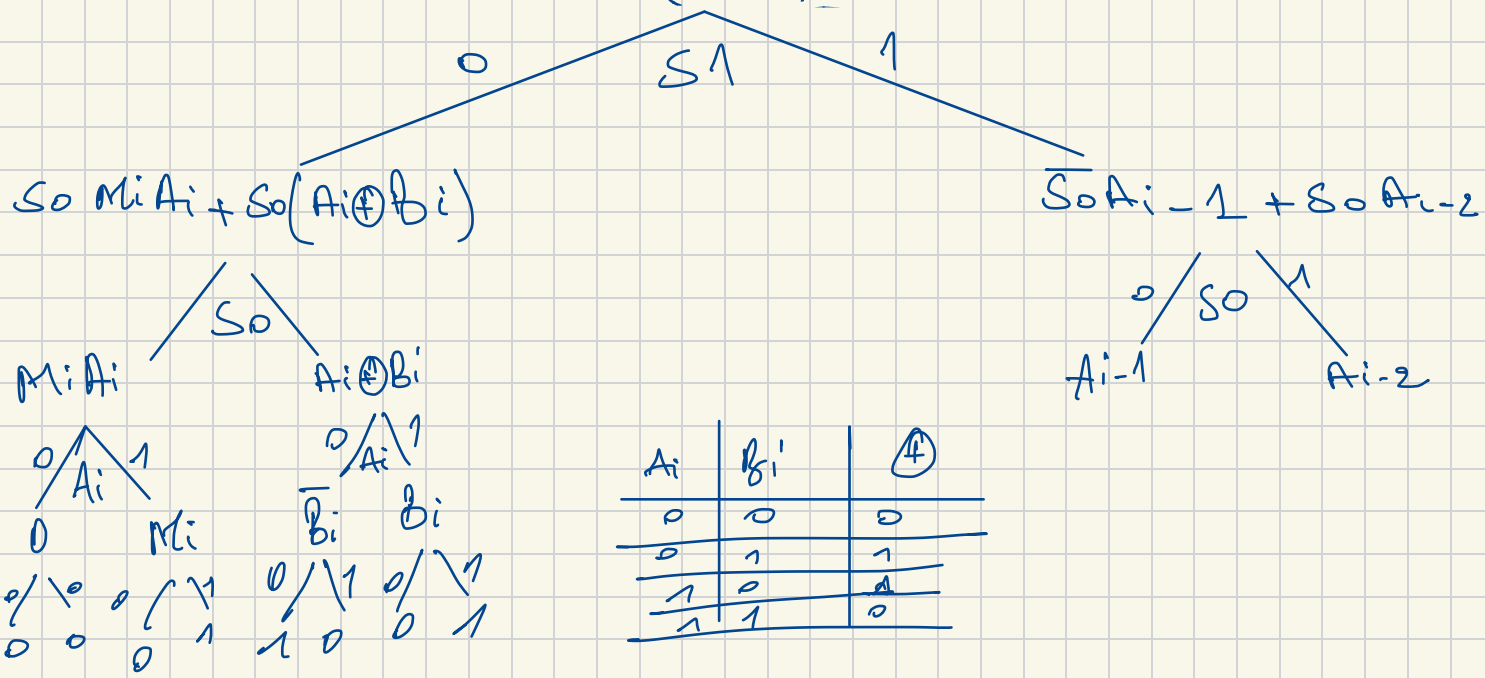
$$S_i = S_1 S^* + \overline{S_1} S^{**}$$

3 cells

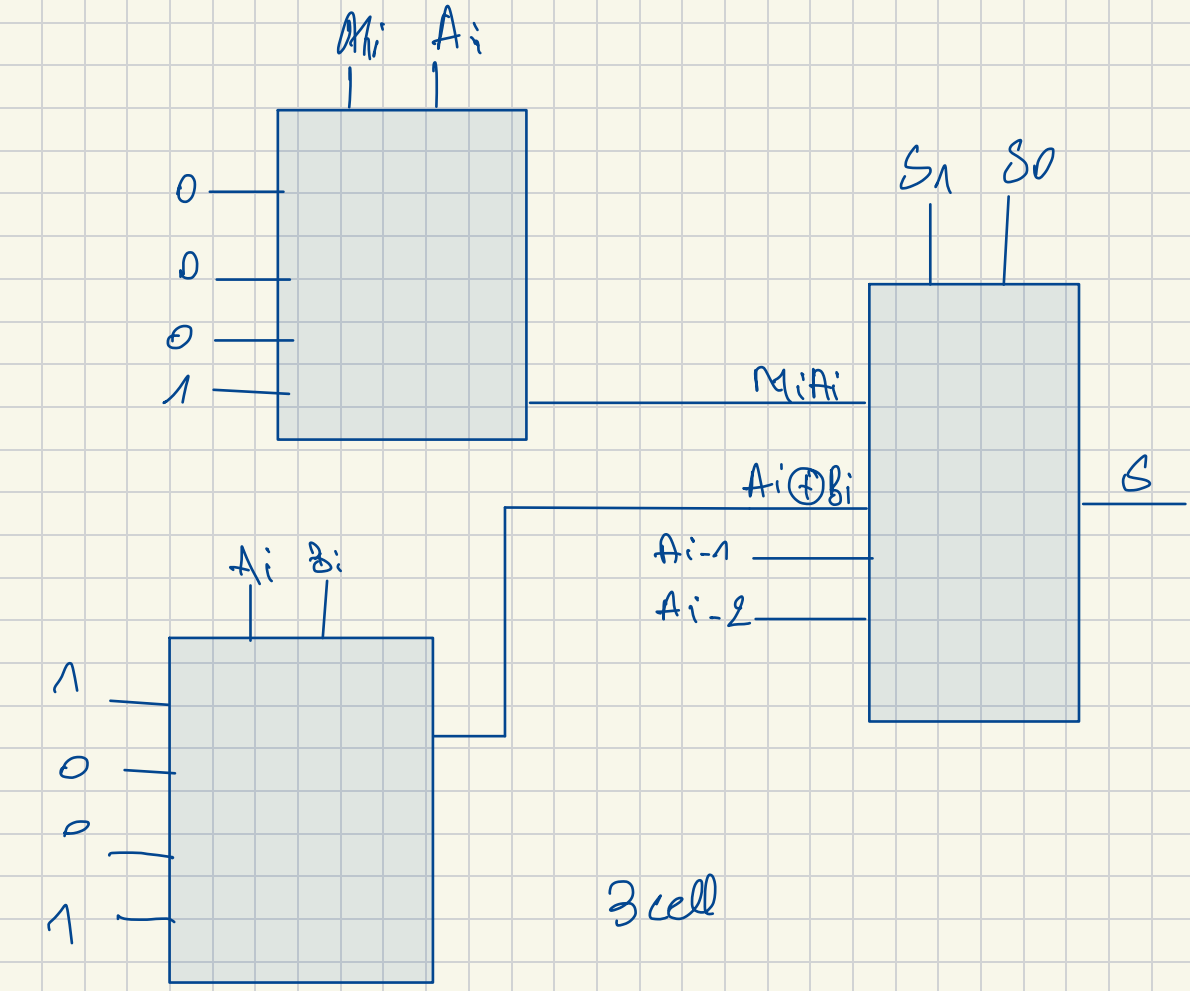
T = 3 ns

MPx 4 → 1

$S = \overline{S1} \cdot \overline{S0} \cdot Mi \cdot Ai + \overline{S1} \cdot \overline{S0} \cdot (Ai \oplus Bi) + S1 \cdot \overline{S0}\, A_{i-1} + S1\, S0\, A_{i-2}$



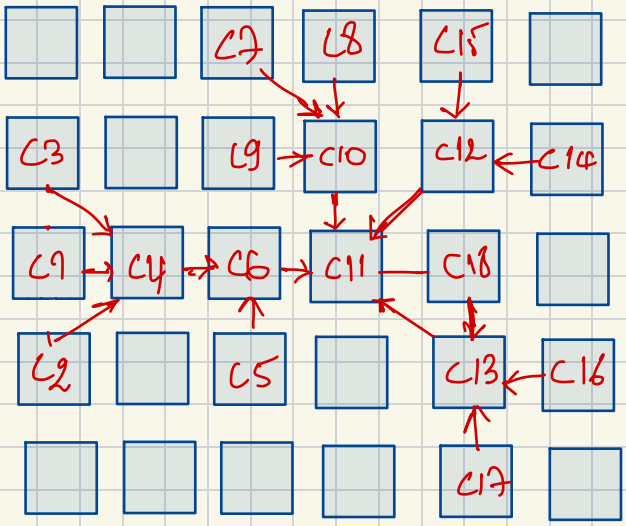
Ai	Bi	⊕
0	0	0
0	1	1
1	0	1
1	1	0



3 cell

T = 2ns

Exo 3 : Routage :



Mesh

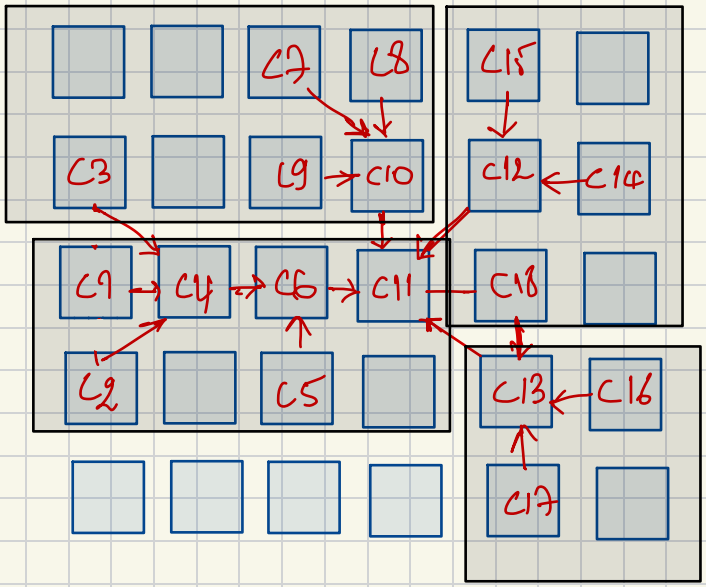
Temps de réponse :

le chemin critique : C1 → C11

ça fait 4 cell sans passer par une Switch Matrix.

$T_R = 4 \times 3ns + 0 \times 1ns = 12ns$

Mesh - cluster :



pareil pour le T_R

$T_R = 12ns$.

4.1 – Lister les process et indiquer les fonctionnalités réalisées

Le système utilise exactement les mêmes mécanismes que dans ton TP2 (affichage VGA + traitement vidéo).
D'après ton rapport, voici les processus **attendus** :

---

1) Processus de division d'horloge

(cf. TP2 — division de 50 MHz → 25 MHz)

- Génère le pixel clock (25 MHz) indispensable au VGA.
- Synchronise tout le pipeline vidéo.

---

2) Processus de balayage horizontal et vertical

(cf. TP2 : pixel_x, pixel_y, end_of_line, end_of_frame)

- Génère les compteurs H/V (0..799 et 0..524).
- Gère la position du pixel dans la trame.

---

3) Processus de synchronisation VGA

(cf. TP2 : hsync, vsync définis par les intervalles)

- Produit hs et vs selon les timings VGA.
- Assure un affichage stable et conforme 640×480@60Hz.

---

4) Processus "zone visible"

(cf. TP2 : enable = '1' si pixel_x<640 et pixel_y<480)

- Active le traitement uniquement dans la zone d'affichage.
- Désactive en dehors (fond noir).

---

5) Processus d'accès mémoire / lecture des pixels

(cf. TP2 : adressage ROM : adresse = pixel_y*420 + pixel_x)

- Calcule l'adresse de la ROM pour récupérer le pixel courant.
- Produit R,G,B pour le pixel affiché.

---

6) Processus de conversion en niveaux de gris

(décrit dans ton rapport comme filtre simple possible)

- Transforme l'image RGB en une image à un seul canal.
- Prépare l'entrée des filtres Sobel.

---

7) Processus du filtre Sobel X et Sobel Y

(cf. TP2 : Partie "Traitement vidéo – idée de conception")

- Nécessite 3 lignes de pixels → *line buffers* (principe donné).
- Applique la convolution 3×3 pour détecter les contours horizontaux et verticaux.

---

8) Processus de recombinaison des deux filtres

(cf. TP2 : "reconstituer l'image à partir des sorties des 2 filtres")

- Combine |Sx| et |Sy| pour produire l'image finale des contours.

4.2 – Peut-on améliorer les performances ? Comment ? Gain attendu ?

Oui.
D'après ton rapport TP2 (partie "traitement vidéo"), les améliorations proviennent de :

✓ 1) Pipeline complet du traitement vidéo

(cf. TP2 : pipeline d'affichage, signaux synchronisés)

- Les étapes (lecture, grayscale, Sobel X, Sobel Y, recombinaison) peuvent être pipelinées.
- Chaque étape utilise l'horloge pixel_clk pour sortir 1 pixel/cycle.

✓ 2) Parallélisation naturelle sur FPGA

Ton rapport mentionne les possibilités d'ajouter des filtres sans changer l'architecture.
Grâce au FPGA :

- Les 9 opérations du masque Sobel peuvent être faites en **parallèle**.
- Le CPU ferait ces opérations **séquentiellement**.

✓ 3) Utilisation de ressources internes

(cf. TP2 : bloc ROM, pipeline RGB → extension possible)

- Le FPGA dispose de **multiplicateurs/DSP** capables d'exécuter les 9 multiplications simultanément.
- Les additions se font en cascade dans les LUT/FF du pipeline.

✓ Gain attendu

Un filtre Sobel = 9 multiplications + additions → séquentiel sur CPU → lent.
Sur FPGA → **1 pixel par cycle**.

Gain typique :
➡️ **×10 à ×20** selon la fréquence et le CPU de référence.

Version courte à mettre dans la copie (basée 100 % sur ton TP)

4.1
Les processus sont :
division d'horloge, compteurs H/V, synchronisations VGA, zone visible, adressage ROM, conversion en niveaux de gris, filtres Sobel X et Y (avec line buffers), recombinaison finale.
Ces processus assurent l'acquisition du pixel, la formation de la fenêtre 3×3 et la génération de l'image de contours.

4.2
Oui. Les performances sont améliorées grâce à une architecture parallèle et pipelinée : les multiplications du masque Sobel sont exécutées simultanément et chaque étape du traitement produit 1 pixel par cycle.
Cela donne un gain d'un facteur **10 à 20** par rapport à un traitement séquentiel sur processeur.

Université Cotes d'Azur/EUR DS4H **Octobre 2024**
Master 1 Electronique
Enseignement : Composants logiques programmables
Documents de cours/TD autorisés

CONTROLE INTERMEDIAIRE(30 MN)

Question de cours

1.1 – Dans quelle situation souhaite–t-on concevoir un système électronique qui intègre un composant logique programmable ?

1.2 - Quel est le rôle d' un composant logique programmable ?

1.3 - Lister les différentes étapes pour réaliser la synthèse de circuit sur un composant logique programmable.

1.4 - Pour déterminer le nombre de cellules élémentaires (CE) : citer le paramètre imortant et les approches utilisées dans le cas des CPLDs, des FPGAs avec LUT puis avec MPX.

1.5 – Expliquer comment est calculé le temps de réponse sur un FPGA. Illustrer par un exemple.

.

1-1- Quand on a besoin d'un système rapide, parallèle, reconfigurable et adaptés à des fonction spécifique
FPGA => haute perf (vidéo, DSP, Protocoles complexes)
CPLD => Gestion d'E/S ou contrôle

2) Le rôle d'un composant logique Prn: réaliser une fonction matérielle configurable, rapide adaptée au besoin en permettant d'implementer des architectures logiques personnalisées, parallèles et reconfigurables => accélérer le traitement par rapport à une exécution processeur.

Je reste sur place 2022.

Université Cotes d'Azur/EUR DS4H
Master 1 Electronique
Enseignement : Composants logiques programmables
Documents de cours/TD autorisés

Novembre 2023

CONTROLE (1H30)

Exercice 1 : Question de cours

1.1 – Le rôle d'un FPGA est de réaliser des accélérateurs matériels. Que cherche-t-on à améliorer par rapport à une exécution sur un processeur ?

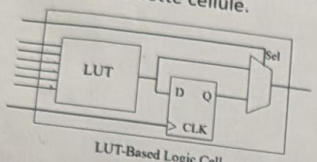
1.2 - Lister les différentes étapes de la synthèse sur un composant logique programmable.

1.3 - Pour déterminer le nombre de cellules élémentaires (CE), citer les approches utilisées dans le cas des CPLDs, des FPGAs avec LUT puis avec MPX.

1.4 – Expliquer comment est calculé le temps de réponse sur un FPGA.

Exercice 2 : Etude des cellules élémentaires

On donne ci-dessous l'architecture d'une cellule élémentaire de FPGA. Indiquer les caractéristiques principales de cette cellule.



LUT-Based Logic Cell

Exercice 3 : Architecture parallèle sur FPGA

On souhaite réaliser sur FPGA un système de détection de contour sur une image. Pour y parvenir, il est nécessaire d'appliquer un filtre (connectivité de 4) dont l'équation est la suivante :

Inew(x,y)=(h1.I(x-1,y)+ h2.I(x,y-1)+h3.I(x,y)+h4.I(x,y+1)+h5.I(x+1))/(h1+h2+h3+h4+h5) avec hi comme coefficients du filtre.

Quelle architecture peut-on proposer pour que les performances de ce système soit supérieure à une exécution sur processeur ? Préciser le gain théorique attendu.

Exercice 4 : Synthèse sur FPGA

On se propose dans le cadre de cet exercice de réaliser une unité de calcul (32 bits) pour la cryptographie intégrant les fonctions : application d'un masque M sur l'entrée A ou B, des décalages. L'équation par étage pour la sortie S_i est donnée ci-dessous :

$S_i = /Sel_1./Sel_0.M_i.A_i + /Sel_1.Sel_0.M_i.B_i + Sel_1./Sel_0 . A_{i-1} + Sel_1.Sel_0 . A_{i-2}$

4.1 -Calculer le nombre de cellules nécessaires pour effectuer la synthèse de ce composant pour les familles suivantes : CPLD MAX7000 (8 produits), FPGA avec 4-LUT puis MPX4->1.

4.2 - Calculer avant routage le temps de réponse : Tcellmax7000 = 3ns, Tcell_4LUT = 1.5 ns, Tcell_mpx= 1ns.

Exercice 5 : Routage

Considérons dans un premier temps un FPGA non hiérarchique

5.1 – Expliquer le rôle du placement et du routage dans la synthèse sur FPGA.

5.2 – Lors de ces phases de placement et de routage, que cherche-t-on à optimiser ? Ces deux phases peuvent elles être réalisées séparément ou de manière conjointe ?

5.3 – Donner deux exemples de routage :

- un dont une communication est à temps nul
- un autre dont une communication est à temps non nul

5.4 – Que devient le placement routage lorsque le FPGA est hiérarchique (cluster de cellules) ?

Exo1 : Feuille 2

Exo2 :

1- LUT : Look up table :

Permet d'implémenter n'importe quelle fonction logique à 4 entrées.

Mémoire SRAM interne $2^4 = 16$

Programmable via bit stream

2- Bascule D :

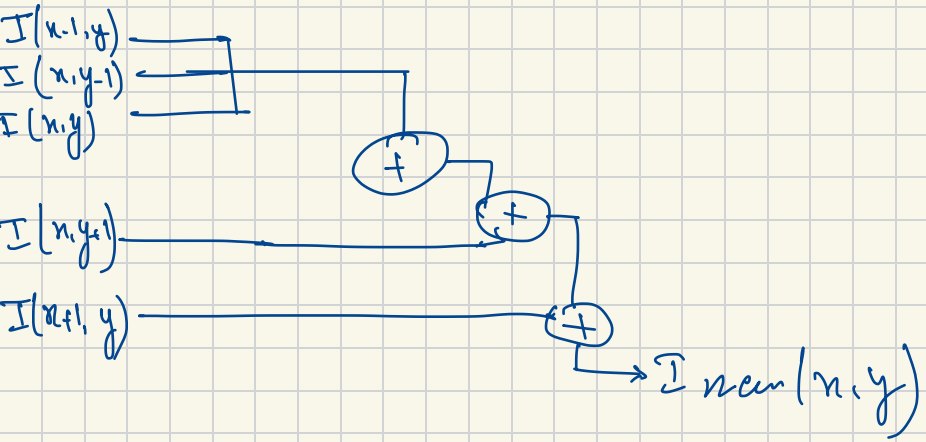
Permet un fonctionnement séquentiel

Peut être contournée : la cellule peut fonctionner → mode combinatoire
                                                  → mode séquentiel

3- MUX de sélection → sortie combinatoire LUT
                    → sortie séquentielle de la bascule D

4- cellule entièrement programmable → combinatoire
                                    → séquentiel

Exo3:



on utilise l'architecture:

Deux line buffers pour disposer en parallèle des lignes y-1 et y+1

- Une fenêtre en croix pour extraire simultanément les 5 pixels

3) 5 Mult réalisés en parallèle avec maths

4) Une chaine d'addition (pipeline) pour produire Inew à chaque cycle d'horloge en additionnant les 5 produits

FPGA calcule toutes les MULT en // et les + en pipeline

⇒ pixel filtré par cycle bien plus rapidement qu'un processeur séquentiel.

Exo4 : Routage :

Placement : Décider on placer les LUT dans le FPGA.

Routage : Décider comment les relier via les switch box et interconnexion

rôle : - réduire le temps de propagation

- réduire la consommation

- minimiser la longueur des fils

- respecter les contraintes temporelles (chemin critique)

À optimiser : chemin critique

temps de propagation total.

distance entre cellules fortement connectés

Séparés ou conjoints ?

théoriquement séparés

En pratique : fortement couplés

- un mauvais placement empêche un bon routage.

- un bon placement réduit le routage.

les deux phases peuvent être réalisées ensemble pour optimiser le chemin critique

5-3- Exemple de routage :

à temps nul :

- connexion directe entre 2 cellules voisines dans le même cluster.

→ 0 SM traversée

T = ons (négligeable).

à temps non-nul :

connexion passant par une ou plusieurs SM

Chaque SM à un temps T_{SM}

temps non nul.